

01b – Securing Information Systems

SWS2 – Overview / ZHAW / Tellenbach

Securing Information Systems

Information System (ISO 27000)

Set of applications, services, IT assets & info-handling components. Involves:

- **Methods & procedures**
- **Data**
- **Technology** (software/hardware)
- **People**

Two fundamental questions

- How to **manage security** so the system stays secure?
- How do we **know** we are secure? (measuring)

ISMS (Info Sec Mgmt System)

- **Systematic approach** to managing info so it stays secure.
- Covers people, processes, IT via a **risk management process**.
- Risk kept at an **acceptable level** through a coherent set of **security controls**.
- Examples: *ISO 27000-series, NIST RMF, BSI IT-Grundschutz, CIS Controls*.

Security Controls

Safeguards/countermeasures to **avoid, detect, counteract, minimize** risks to assets. Characterized by **attributes**:

- **Info-sec property**: Confidentiality, Integrity, Availability (CIA)
- **Category**: people, physical, technology, organizational
- **Function**: identify, protect, detect, respond, recover

Type of a Control (!)

- **Preventive**: prevent incident (auth, firewall)
- **Detective**: identify incident in progress (alarm, IDS)
- **Corrective**: limit damage / recover (backup, redundancy)

Controls in ISMS – Abstraction

- ISMS operate at **high abstraction** (apply to all orgs/sectors).
- Different granularity; sector-specific control sets exist.
- Little guidance on **technical** implementation.

Example: ISO 27000 ("27k")

- **ISO 27001:2022**: 93 controls in Annex A (For Example): Network Security: "Networks and network devices should be secured, managed and controlled to protect information in systems and applications." So it says what to do, but not how to do it.
- **ISO 27002:2022**: adds **implementation guidance** – control objectives & best-practice controls.
- Not all controls relevant to every org; industry extensions exist (e.g. telco ISO 27011).

ISO 27002 control documents have: *Control, Purpose, Guidance* + attribute table.

Ex: Control "Protection against malware"

ISO 27002:2022 tags each control with 5 **attributes**; possible values per attribute below, **highlighted = the ones that apply** for this control:

- **Control type**: Preventive / **Detective** / Corrective
- **Info security properties**: Confidentiality / **Integrity** / Availability
- **Cybersecurity concepts**: Protect / **Detect**
- **Operational capabilities**: System_and_network_security / Information_protection
- **Security domains**: Protection / **Defence**

Implementing an ISMS

- Task for **management** – usually the **CISO** (talks to board in business/risk terms).
- Gives the **big picture**; standards act as a **checklist**.
- **Certification**: checks whether company implements a standard.
- Complex, costly; **no guidance to efficiently prioritize** vs most common threats ⇒ **CIS Controls**.

CIS Critical Security Controls

Overview

Is a list of the most important security measures to defend against real cyber-attacks

In comparison to ISO 27000, ISO is high-level and says "what" to do, but not "how" to do it. CIS Controls are more specific and actionable, with detailed implementation guidance.

- Best-practice guidelines, started **2008** (US defense breaches).
- **18 controls** for current cyber attacks; by experts.

Guiding principles:

- Only **critical** controls, based on **attacker behavior** & data.
- **Prioritize as a community**; immediate action > completeness.
- Provide info on how to implement & **automate**.

The 18 Controls

01 Inventory Enterprise Assets • 02 Inventory Software Assets • 03 Data Protection • 04 Secure Config • 05 Account Mgmt • 06 Access Control Mgmt • 07 Continuous Vuln Mgmt • 08 Audit Log Mgmt • 09 Email & Web Browser • 10 Malware Defenses • 11 Data Recovery • 12 Network Infra Mgmt • 13 Network Monitoring & Defense • 14 Security Awareness & Training • 15 Service Provider Mgmt • 16 Application Software Sec • 17 Incident Response Mgmt • 18 Penetration Testing

Safeguards

Safeguards = concrete things to implement (called sub-controls before v8). Each control has N safeguards split across IG1/IG2/IG3.

CIS Control content

- **Overview**: purpose & defensive use
- **Why important**: how attackers exploit absence
- **Processes & tools**: technical impl./automation
- **Safeguards**: table of specific measures

Prioritization (!)

- **Until v7.1**: controls 1–6 = "cyber hygiene" first. Criticized as too simplistic.
- **Since v8**: prioritization via **Implementation Groups IG1–IG3**. Control order still has value (e.g. 1&2 protect vs malware better than 10).
- In practice: baseline + **risk assessment** (cost-benefit per environment).

Implementation Groups (IG)

- **IG1 – Basic Cyber Hygiene**: limited expertise, COTS, small biz/home; thwarts **general, non-targeted** attacks. Stop here if SME, low-sensitivity data, focus on operations.
- **IG2** (incl IG1): copes with operational complexity; enterprise-grade tech & expertise; multiple depts, compliance, sensitive data, public trust.
- **IG3** (incl IG1+IG2): security experts; vs **targeted attacks**, reduce impact of **zero-days**; regulated, availability+CIA at heart of business.

Safeguards per IG – "x/N" notation (!)

Each control has N safeguards; IGs are **cumulative** (IG2⊇IG1, IG3⊇IG2). **"x/N"** = how many of the N safeguards apply at that level. *Ex: Control 09 Email & Web Browser (7 safeguards) → IG1 2/7, IG2 6/7, IG3 7/7:*

- **IG1 (2/7)**: only supported browsers/mail clients + DNS filtering (cheap, high-impact).
 - **IG2 (6/7)**: + URL filters, restrict extensions, DMARC, block file types.
 - **IG3 (7/7)**: + email-server anti-malware.
- IG1 = 0/N** (e.g. 13 Net Monitoring 0/11, 18 PenTest 0/5) ⇒ basic-hygiene org does **nothing** in that control yet.

Selected Controls (detail)

- **CSC 1 – Enterprise Assets**: active+passive device discovery, asset inventory DB, NLA (802.1x), PKI, analytics. *Why both active+passive?* active=fast/triggers, passive=continuous/finds hidden & unauth devices.
- **CSC 2 – Software Assets**: software whitelisting, inventory tool, NAC to **enforce**. Very effective vs malware (blocks unauthorized SW).
- **CSC 7 – Continuous Vuln Mgmt**: SCAP vuln scanner + patch mgmt + analytics.

Summary & Criticism

- Most companies implement only a **subset**.
- Protecting IS is complex: needs checklists, resources, risk mgmt.
- Critique: paper "A Critical View on CIS Controls" (S. Gros) – some valid, some academic.

Measuring Security of IS

How secure is your org?

Factors (incomplete): **people, physical security, vulnerabilities (known+unknown), controls, configuration & architecture, processes & policies**. **Measuring security scientifically/holistically is (most likely) not possible**. Same for measuring **risk**.

Approaches to measure

- **Security: Audit** (systematic eval vs established criteria), **penetration test**, measures of ISMS performance & effectiveness.
- **Risk: Risk = Likelihood × Impact**; estimating likelihood is very hard.

Why measure at all?

- Identify controls **implemented incorrectly / ineffective / missing**.
- **Quantify progress** (e.g. time to fix vuln, phishing resilience).
- **Compliance** evidence (ISO 27001 requires eval of performance & effectiveness).
- Support **risk-informed decisions**.

Measure frameworks

NIST SP 800-55 & ISO 27004 provide measure templates (Goal, Measure, Formula, Target, Frequency, Responsible parties, Data source, Reporting format). E.g. % of high vulns mitigated within defined time; trend of detected-but-not-blocked malware attacks.

Recommendations

- Measuring whole infrastructure safety hardly possible, but we must measure to **decide** ⇒ aim: **reduce uncertainty**.

- **Question** what/how is measured & what conclusions (e.g. "2× failed SSH logins" $\neq$ 2× exposure).
- Use **comparative measurements** (IDS subnet > no IDS; static analysis → more secure).
- Measure **progress & maturity** with best-practice measures.

Take-away: Securing information systems is hard. Measuring security is hard.

02 – Threat Landscape

SWS2 – ZHAW / Tellenbach

Threat Landscape – Basics

Definition

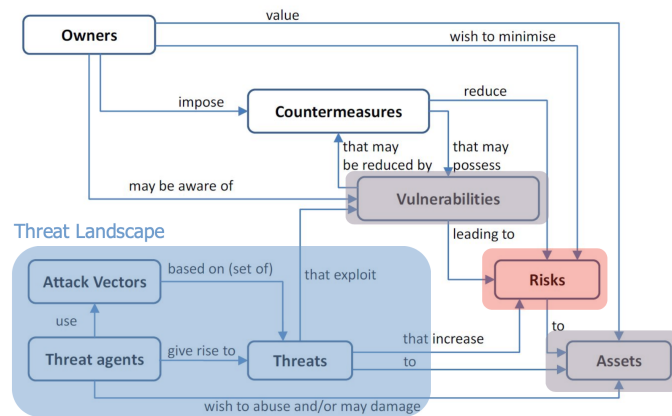
A collection of **threats**, **threat actors (agents)** and **observed trends**. Tracking it = knowing:

- threat agents & their **capabilities**
- **weapons** and **tactics** used
- which threats are **most relevant**
- **emerging** threats & actors

Why: know your enemy / prepare; motivates **investment** in controls.

Elements of Risk (relationship)

Threat agents use **attack vectors** → give rise to **threats** that exploit **vulnerabilities** → lead to **risks** to **assets**. Owners impose **countermeasures** to reduce risk. Threat Landscape = threat agents + attack vectors + threats.



Importance for Risk

Risk = Likelihood × Impact; Risk = Threat × Vulnerability × Consequence. The threat landscape is **external / out of your hands** and drives the likelihood part.

Important Elements

- **Threat actors:** state actors, cyber criminals, insiders... Attributes: **motivation**, **skill/resources**, **organization**, **tactics**.
- **Threats:** type+description (ransomware, phishing...), **targeted assets**, **attack vectors** (methods/tools, split in phases → kill chain).
No standardized way to characterize; lines between actors/types often blurred; companies define own vocabulary.

Threat Actors

Profile parameters

Motivation (intel/money/destruction/fame), **Resources** (low/med/high), **Skill** (low/med/high), **Role:** Developer / Provider / Operator / User (**Dev/Pro/Op/User**).

Cybercrime

- **Profit** from illegal activity; optimize **ROI** (max money / min effort+risk). Mot: Money; Res: Med-High; Skill: Low-High; Roles: User+Dev/Prov/Op.
- **Professionalized industry** – division of labor via ***-as-a-Service** (below).
- **Ransomware** (top tactic) – extortion levels:
 - **Single:** encrypt data, sell decryption key (backups defeat it).
 - **Double:** also **exfiltrate** & threaten to leak (backups don't save you).
 - **Triple:** + DDoS / pressure customers & partners.
- Delivery: **automated** (masses) vs **human-operated = Big Game Hunting** (high-value targets).
- Other tactics: **social engineering**, exploit **WFH/remote access**, low-hanging fruit (SMEs, bad cloud), attack **MSPs** (1 hit → all their customers).
- Challenges: **cross-border** (diff. legal frameworks), takedowns not sustainable (infra disposable).
- *Ex: DarkSide RaaS / Colonial Pipeline (2021)* – billing crippled → 6-day shutdown, fuel shortage, \$5M BTC; FBI recovered 63.7 BTC.

*-as-a-Service ("anything"-aaS)

Attack capabilities **packaged & rented** like cloud products (SaaS-style) ⇒ **lowers barrier to entry** (no skill needed, just buy). Often w/ updates, **AV evasion**, support, SLAs.

- **Roles:** Developer (builds) / Provider (infra/access) / Operator (runs) / User (buys & aims).
- Types: **RaaS** (ransomware), **AaaS** (access), **PaaS** (phishing), + infection/spam/DDoS.

Prices: \$8 iTunes creds • \$40 20k spam • \$100 infection 1k installs • \$1000 DDoS 1 month • \$40k malware+bootkit • \$400k+ GovWare

Nation States / State-Sponsored

- Act in interest of a state, in **secret**. Mot: strategic (espionage); Res: High; Skill: High; Roles: User+Dev/Op.
- Targets: state/military secrets, critical infrastructure.
- Tactics: **long-term**, **novel**, **sophisticated** – zero-days, **APT**, supply chain, ICS, AI, hack-and-leak.
- Trend: also act for **personal gain** (blurs espionage/crime).
- Challenges: legal for them; **sanctions & name-and-shame**; use **false flags** & standard tools.
- *Ex: SolarWinds (SUNBURST)* – trojanized Orion updates, supply-chain, attrib. Russia (APT29), discovered by FireEye 12/2020.

Hacker-for-Hire

- Companies offering **offensive cyber capabilities** (vuln research, exploitation, malware, C2, mgmt, support). Mot: Work/Job; Res: High; Skill: Professionals; Roles: Dev/Pro/Op.
- Bundled services: AaaS, RaaS, PaaS.
- Clients usually governments → **plausible deniability** (proxy).
- Operate **legally** in their country; hard to predict.
- *Ex: NSO Group / Pegasus spyware* – SMS links, network injection, **zero-click**; spies on journalists/critics; lawsuits (WhatsApp).

Hacktivist

- Individuals/loose groups (e.g. Anonymous) for **social/political change**. Mot: Work; Res: Low*; Skill: Low-Med; Roles: User.
- Tactics **old school:** (**D**)DoS, **defacements**, release sensitive data.
- Hard to predict – **event-driven**; crowd-sourcing = huge leverage.
- *Ex: Tillie Kottmann / APT 69420* – Verkada hack (150k cams) via exposed creds, Google dorking.

Insider

- **Unintentional** (lax procedures → address w/ **DLP**) or **intentional** (steal/leak/sabotage). Mot: extortion/revenge/profit; Res: Low; Skill: Low-Med; Roles: User.
- Challenge: **legitimate access** + know the protections.
- Important: spot unhappy employees, close knowledge gaps, monitor unusual activity, limit access.

Others

- **Script Kiddy:** runs tools without knowing; *-aaS boosts activity. (*Can do SQLi via tools – yes.*)
- **Cyber Terrorists:** disruption for alarm/panic; no commonly accepted definition; no major event yet.

Threats

Knowing threats – why?

Roadmap for future (policy/biz/research); learn new attack vectors. Helps **CEO/CIO** (risks to business), **CISO** (trainings, new/modified controls), **SecOps** (patterns, indicators).

Threat Taxonomy

Classification of threat types at various detail → common language. Many proposals (no one-fits-all): *Open Threat Taxonomy*, *ENISA*, *NIST*, *Operational Cyber Sec Risks*, *Cambridge Risk Framework*. *Open Threat Taxonomy categories:* Physical, Resource, Personnel, Technical threats (w/ threat rating).

Sources of Information

Long term (strategic/tactical)

Threat reports few times/year: **ENISA ETL**, Fortinet, Bitdefender, Proofpoint, McAfee. Some describe threats only (not whole landscape).

Short term ("real-time")

- News portals (Heise Security, DarkReading), security mailing lists.
- **Threat intelligence providers:** advisories + **actionable info** (CERTs, OTX feed), real-time web pages.
- **Threat Intelligence Platforms:** aggregate/correlate, **machine-readable** data → IDS/IPS/SIEM.
- Exchange formats: **STIX** (Structured Threat Info Expression), **TAXII** (Trusted Automated eXchange of Indicator Info).

APT & Cyber Kill Chain

Advanced Persistent Threat (!)

- **Advanced:** advanced tech/techniques to penetrate defenses (own/unknown vulns); develop tools as needed.
- **Persistent:** maintain **long-term** access, keep trying, **hide** from detection until objective reached.
- **Threat:** coordinated **human** actions; skilled, motivated, organized, well-funded with a specific objective.
- Bad: median **243 days** attacker present before detection.

Cyber Kill Chain (Lockheed Martin) (!)

Identifies what adversary must complete; **intrusion-centric** (not all attacks fit). **Defender advantage**: attack succeeds only if all steps succeed → defenders disrupt at **any** step.

Preparation:

- **1 Reconnaissance**: gather public info on target.
- **2 Weaponization**: build exploit + malicious payload.

Intrusion:

- **3 Delivery**: send payload (email/USB/web).
- **4 Exploitation**: exploit vuln to execute payload.
- **5 Installation**: install malware (can be in-memory).

Active Breach:

- **6 Command & Control (C2)**: remote channel (generic, throughout attack).
- **7 Action on Objectives**: achieve goals inside network (months, 100s of steps).

Summary

- Many threat agents, different motivations/tactics/skill/resources/roles.
- Orgs must consider which agents are **most relevant**.
- Kill chain → kill attack at any step; APT ⇒ you're hacked.
- **Not a fair game**: attacker finds **one** hole, defender must fix **all**.

03 – Security Controls: SIEM

SWS2 – ZHAW / Tellenbach

Detecting a Security Problem

The data problem

Controls (firewalls, AV, IDS, vuln scanners...) all **produce data**:
■ **Preventive** controls → data on an attack that **failed** (e.g. login).
■ **Detective** controls → data on an attack **just happening** (e.g. IDS).
■ **Corrective** controls → data on **corrective actions** (e.g. DDoS offloading).
Volume: mid-size co. = **millions of events/day**; large co. = **billions/day**; netflow huge. ⇒ **information overload**: find THE **needle in a stack of needles**.

Two challenges → two tools

- 1st: **Log management** (have & handle the logs).
- 2nd: **Make sense** of security-relevant log data.
- Both addressed by a **SIEM** (or separate log mgmt + SIEM).

Log Management

Before making sense of logs

- You need to **have logs!** Solidify the **logging process**: what is logged where/why, how long & where stored.
- Consider **legal aspects** – behavioural tracking can be problematic. Solutions help with: **collection, retention, indexing/parsing & searching, reporting** (all log types). *Tools: Logstash, Graylog, Splunk, AlienVault, LogRhythm.*

Architecture (3 tiers)

- **1 Log generation**: hosts/services/apps that create logs.
- **2 Log analysis + storage**: log servers receive/copy logs from tier 1.
- **3 Log monitoring**: dashboards to review data + automated-analysis results.

Real-time, near-real-time or batched; today often **scalable cloud** architectures.

SIEM

What it does

Security Information & Event Management: gathering, aggregating, **correlating**, analysing, **prioritizing** & presenting info from **disparate systems**. The **"E"** = correlates **events**, raises & prioritizes alerts. *Funnel*: 3M total events → 15k correlated → 24 high-prio alarms → **8 true incidents**.

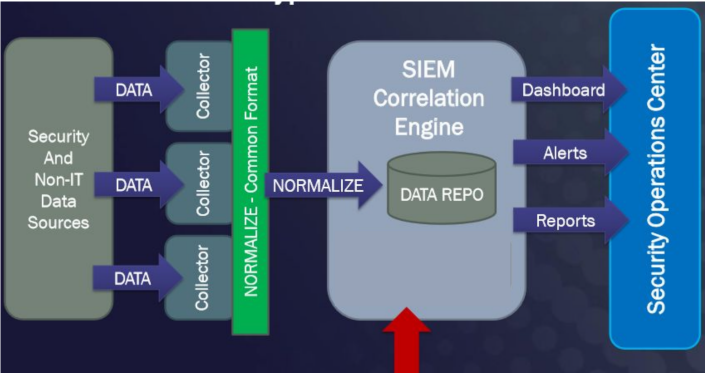
Usage scenarios

- **SOC**: analysts 24/7, chase alerts.
 - **Mini-SOC** / "morning after": 1/24, review & drill down.
 - **Automated SOC**: configure & forget, alerts only.
 - **Compliance reporting**: review weekly/monthly.
- Stealthy/advanced attacks ⇒ SIEM must learn what's **normal (baseline)** & flag the **exceptions (anomalies)**.

Architecture & key components

Sources → **Collectors** → **Normalize** (common format) → **Correlation Engine** + data repo → Dashboard/Alerts/Reports → **SOC**. *Commercial: IBM QRadar, HP ArcSight, Splunk. Open-source: AlienVault OSSIM, Wazuh, SIEMonster.*

- **Asset directory**: asset value for **risk calc/prioritization**.
- **Vulnerability scanner**: correlate host alerts with its known vulns.
- Sources: firewall/auth/system logs, availability monitoring, AV, honeypots, **HIDS/NIDS**.



IDS / IPS

Intrusion Detection System (IDS) vs Intrusion Prevention System (IPS)

IDS vs IPS

- **IDS**: monitors network/system activity to **detect** attacks & abuse (detect + log only).
- **IPS**: adds a component to **prevent/mitigate** the detected attack.
- IPS risk: a **false positive** can **lock out legitimate users** (e.g. block a customer); some damage may already be done by detection time.
- Two types: **Host-based** (HIDS/HIPS) & **Network-based** (NIDS/NIPS).
- Components: host/network **sensors**, central DB, **correlation engine**, mgmt console.

HIDS / HIPS

- **HIDS**: sensor software **on the monitored host**; watches system calls, app logs, file modifications, processes, host network data. *Ex: OSSEC, AIDE, Samhain, Fail2Ban, Sagan.*
- **HIPS** actions: quarantine files, restore from storage, suspend accounts, update firewall (block IP), start/kill processes, shut down.
- *Ex: Fail2Ban* – scans logs (e.g. apache error_log), detects malicious IPs (too many failed logins / probing), updates firewall to reject IP for a time.

NIDS / NIPS

- **NIDS**: sensors are **devices in the network** seeing the traffic (switch monitor port, network tap, security gateway); analyses traffic for malicious/suspicious activity. *Ex: Snort, Bro, Suricata, Sagan.*
- **NIPS**: traffic must **flow through** a monitor/block component. On event: traffic blocked temporarily → rule engine decides alert; no alert = forward, alert = block (log & discard).

Suricata rule (example)

- ```
alert tcp $EXTERNAL_NET any -> 10.200.0.0/24 80 (msg:"...root.exe access"; ...; sid:1255;)
■ $EXTERNAL_NET: all IPs not in $HOME_NET.
```
- **\$HOME\_NET**: user-defined ranges (default 192.168/16, 10/8, 172.16/12).
  - **any**: from any source port; -> = connection to.
  - **msg**: text shown in the logged alert.

Measuring IDS performance (!)

**FPR** = FP/(FP+TN) – "ratio of **benign** points classified as attack".  
**FNR** = FN/(FN+TP) – "ratio of **attack** points classified as benign".  
benign = gutartig  
**Precision** (= "alert quality", slide calls it TPR) = TP/(TP+FP) = how often an alert is a **real** attack. *Marketing: FPR 0.1%, FNR 0% (misses no attack) – sounds great!*

**Ex (Scenario 1)** – 500k pts/day, 0.1% are attacks. **Two different 0.1% on two pools**:

- **TP = 500** = 0.1% of **all** 500k = the **real attacks** (all caught, FNR 0).
- **FP ≈ 499** = FPR 0.1% of the **499.5k benign** pool = **false alarms**.
- Alerts = TP+FP = 999 → precision = 500/999 ≈ **50%** are real.
- **Scenario 2**: 10M pts/day, 0.006% attacks → TP=600, FP≈9999 → precision ≈ **5.6%**. ⇒ **False Positive Paradox / base-rate fallacy**: attacks are **rare**, so a tiny FPR on the **huge benign pool** produces ≥ as many false alarms as real attacks. When **attack rate < FPR**, FPs dominate. A "99.9% accurate" IDS can still give mostly junk alerts.

Strengths & limitations

- **+** detects known patterns (SQLi, scans, malware spread, anomalies); IPS stops attacks.
- **-** must be **tuned** for low FPR/FNR; NIDS **can't see encrypted** traffic; IDS doesn't stop attacks (needs humans); IPS must buffer/delay packets (resource-heavy, limits correlation); IPS FPs lock out users.

Event Correlation

Concept

**"Establish relationships between events"**: combine multiple isolated events into **one relevant incident** – "if this, followed by that, take action". *Ex: Nessus finds Heartbleed on host X, then IDS alerts suspected Heartbleed attack ⇒ raise alert.* Compares parameters: src/dst IP, user/machine, time. **Not** reliant on a single signature/rule ⇒ powerful attack-pattern analysis from **multiple independent inputs**.

Baseline & anomaly (!)

**Baseline** = what is normal; **anomaly** = exception. *Ex: Sally (financial analyst) logs into AD Mo–Fr 08:30–08:39; suddenly SSH to DNS server + logins at 01:31/06:02 weekend ⇒ anomaly. Ex: badge "exit building" 16:00, then login to perimeter server 16:02 ⇒ impossible. ⇒ correlation + **clean baselines** are vital.*

AlienVault SIEM processing

Normalized events → evaluated against **policies** (= conditions + consequences):

- **Conditions**: which events a policy processes.
- **Consequences**: what happens (send email, forward/modify/create/drop event, alter priority/reliability).

Risk assessment formula (!)

**risk = asset × (reliability × priority / 25)**

- **Asset value** 0–5 (asset importance).
- **Priority** 0–5 (based on event type).

- **Reliability** 0–10 (is it really an attack?).
  - **Alarm if risk  $\geq 1$** . Higher risk = examine first.
- Vuln scans (OpenVAS) become event sources; severity  $\rightarrow$  reliability.

Correlation vs Cross-correlation

- **Correlation**: rules associate multiple events from the **same data source**. *Ex: brute-force = correlate failed + successful logins from same IP; rising reliability with count; limit to bad-reputation hosts.*
- **Cross-correlation**: events from **different data sources**. *Ex: correlate IDS alert with vuln info  $\rightarrow$  if target is vulnerable, set reliability to max.*

(Threat) Use Cases

What & why

A **use case** = a **logical, actionable, reportable** component: a (set of) **rule(s), report, alert** or **dashboard** solving a need. *Ex: detect outbound spam; weekly vuln report; dashboard "logins from bad country".* **Heart of any SIEM deployment** & very **time-consuming** – SIEMs **don't work out of the box**. A use case is "developed" like a mini-project.

Stages (Spam example)

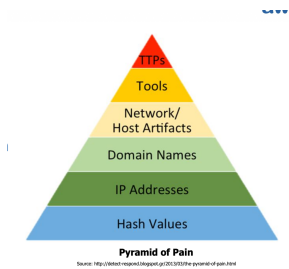
- 1 **Requirements** (business/compliance/regulatory/security)
- 2 **Scope** (relevant IT infra)
- 3 **Event sources** (HIDS/NIDS, mail filters, network/endpoint events)
- 4 **Validation** (data available & parsed)
- 5 **Logic** (signature/behavior, e.g. 10 port-25 conns/min)
- 6 **Implement & test** (output: report/notification)
- 7 **Response** (procedures to operate it)
- 8 **Maintenance** (tune, delete if obsolete)

Threat Feeds & IOCs

Threat intelligence

**Machine-readable threat feeds** with **Indicators of Compromise (IOCs)** feed the correlation engine. *Ex feeds: detect outbound traffic to known C2/malware domains (vs proxy/firewall logs), email from known phishing servers (vs SMTP logs), DNS queries to non-standard TLDs, infected host that hasn't triggered AV.*

Pyramid of Pain (!)



Concept – two separate axes (!)

Detecting & responding to an indicator fast = **denying the adversary its use**. Each indicator has **two independent properties**:

- **Accuracy** (good for **you**): if it matches, how sure it's really malicious (few false positives).
- **Pain** (cost to the **attacker**): how hard/costly to **change it** & evade you.

Bottom = trivial for attacker to change, top = must reinvent. *Ex: a hash = high accuracy (matches exactly one file) but **trivial pain** – attacker flips 1 bit / recompiles  $\rightarrow$  new hash  $\rightarrow$  your block is useless  $\Rightarrow$  whack-a-mole. A **TTP** = high pain (attacker must change how they operate) but lower accuracy (legit admins also use e.g. PowerShell  $\rightarrow$  false positives).*

Levels (low  $\rightarrow$  high pain)

- **Hash values** – pain **Trivial**, accuracy High. *Flip 1 bit  $\rightarrow$  new hash.*
- **IP addresses** – pain **Easy**, acc. Med-High. *(Anon) proxies, fast-flux.*
- **Domain names** – pain **Simple**, acc. Med-High. *Must register/host, but lax/anon registrars (crypto) make it cheap.*
- **Network/Host artifacts** – pain **Annoying**, acc. Med-High. *URI patterns, C2 in protocols, weird User-Agent; registry keys, dropped files. Attacker must find why detected & fix it.*
- **Tools** – pain **Challenging**, acc. High. *Custom protocols, fuzzy hashes. Attacker must build a new tool (costs time/money).*
- **TTPs** – pain **Tough!**, acc. Low-High. *How adversary operates (Pass-the-Hash, spear phishing, tailgating). Detecting the behavior forces them to relearn  $\rightarrow$  give up or reinvent from scratch.*

$\Rightarrow$  **Use indicators that cause the most pain whenever possible.** (SIEM/operational  $\rightarrow$  lower levels; strategic threat intel  $\rightarrow$  upper levels.)

# 04 – PenTesting 1: Security Testing

SWS2 – ZHAW / Tellenbach

## Security Testing – Big Picture

### 4 dimensions

- **Why?** find vulns, improve defences, educate/train, compliance, identify attack surface, incident response, improve product security, (unique) selling point.
- **What?** hardware, app/service, security control, humans, processes, systems, organization, supply chain.
- **How?** test types: **static** / **dynamic**, **manual** / **automated**.
- **By whom?** internal (devs, security pros), 3rd party, crowd.

## 6 Security Testing Methods (!)

Each described by: *Purpose, Attacker goal, Assets in scope, Result, Method, Requirement, Frequency.*

### 1. Vulnerability Scanning

- Purpose: identify **well-known** vulns; compliance.
- Scope: source code, apps, systems, whole infra.
- Result: **potential** vulns + generic risk rating.
- Method: **fully automated** (OpenVAS, LGTM).
- Freq: **continuous** (signatures & assets change).
- Req: Needs mature **vuln-management** process (verify/prioritize/fix).

### 2. Classical Penetration Testing

- Purpose: find as **many vulns as possible** + fixing advice; compliance.
- Attacker goal: efficiently find many (mainly **"easy-to-find"**) vulns.
- Scope: **one/few** (single app/system/service).
- Result: **verified** vulns + risk rating + fix advice.
- Method: (semi-)automated tools + standardized procedures (OWASP guide).
- Freq: **1–4×/year**, once per new release.
- Req: Needs mature **vuln-management** process + test environment (e.g. staging with production-like data).

### 3. Red-Team Testing

- Purpose: test the org's **detection & response**.
- Attacker goal: achieve a stated goal **without getting caught** (steal data, become domain admin...).
- Scope: **many/all** (physical, human, cyber).
- Result: goal achieved (y/n) + how.
- Method: goal/scope-dependent, often **social engineering**.
- Req: Needs a **mature security program**.
- Freq: periodically(?).

### 4. Purple-Team Testing

**Blue team** = the defenders (SOC/IR: run monitoring, SIEM, detection rules, respond to incidents). **Security posture** = the org's overall defensive strength – how well it prevents, detects & responds to attacks (not just one bug).

- Purpose: **improve posture**; focus on **detective/preventive** controls; spot malicious actors already in the environment.
- **Red + Blue collaborate** (purple = red+blue): attackers **share each move in real time** so defenders learn to detect it.

- Attacker goal: **help the blue team** catch & learn from them (not to "win" silently).
- Scope: predetermined systems/employees.
- Result: control improvements (e.g. **new detection rules**) + plan for unresolved gaps.
- Method: simulate relevant attack patterns.
- Req: Interfaces and collaboration between incident response, security tooling, net engs. and vuln. mngmn. work well
- Freq: e.g. quarterly.



### 5. Breach & Attack Simulation

- Purpose: improve posture; detective/preventive; uses **prominent attack vectors (MITRE ATT&CK)**; multi-step along the **kill chain**.
- Result: resilience report.
- Method: **automated platform** (SaaS) running scripted multi-stage attacks.
- Freq: **continuous**.
- Req: Needs purple-team prerequisites + big-budget blue team.

### 6. Bug Bounty Program

- Purpose: find vulns, **pay only if found**. **Public** (anyone) or **private** (invite only).
- Attacker goal: earn money/prestige.
- Scope: often single app/service.
- Result: vuln reports.
- Method: per program rules (what's allowed).
- Req: Needs legal framework + willingness to "go public" + accept **untrusted testers**.
- Freq: continuous.

### When to use which

- Find **known** vulns → Vuln Scanning, Pentest, (Bug Bounty).
- Find vulns in self-developed apps → Pentest, Bug Bounty.
- Improve product security → Bug Bounty.
- Improve **defensive measures** → Purple, BAS.
- Test defenses, incident response → Red, Purple, BAS.
- Educate/train blue team → Purple, BAS, Red.
- Compliance (Meaning: meeting regulatory requirements) → Vuln Scanning, Pentest.

*Note: matters whether system is internal / public / 3rd-party-hosted (e.g. public bug bounty ≠ internal systems).*

## Penetration Testing

### Definition

- **NIST SP 800-115**: security testing where assessors **mimic real-world attacks** to find methods of **circumventing security features** of an app/system/network.

- Business view: analyse all/designated parts of the IT environment to **find & exploit vulns** and identify the **business risk**.

**Caveats:** a pentest is a **snapshot**; "no vuln found" with 1 tester / 10 days / limited gear **proves little**.

### Motivation

Find holes **before attackers**; find holes **automated tools can't**; test **detect & respond**; prove issues to **management**; raise **awareness**; verify configs / new tech; close **compliance** gaps (PCI-DSS, HIPAA).

### Focus & scope

- Focus on: hacking IT assets, getting **access to info**, physical security, **social engineering**.
- **Clear but limited scope** (e.g. web-app only from browser's view).
- Success factors: experience, skills, equipment, **out-of-the-box thinking**.

## Pentest Methodologies

- **OSSTMM** (Open Source Security Testing Methodology Manual): tests **everything** (guards → satellites); **mathematical model** for a comparable security level.
- **OWASP Testing Guide** (v4): **web-app** security testing; hands-on advice + tool refs.
- **NIST SP 800-115**: process (planning, analysis, validation) + Linux-tool appendix.
- **PTES** (Penetration Testing Execution Standard): basic principles & steps; industry effort, somewhat outdated/incomplete.

## Pentest Phases (PTES) (!)

Cyclic 7 phases:

- 1 **Pre-engagement Interactions**
- 2 **Intelligence Gathering**
- 3 **Threat Modeling**
- 4 **Vulnerability Analysis**
- 5 **Exploitation**
- 6 **Post Exploitation**
- 7 **Reporting**

## Pre-engagement Phase

### Three pillars

- **Scoping**: define **what** is tested (in-depth single app? over internet w/o employee help? wide IP range to find a way in?).
- **Rules of Engagement**: **how** testing occurs (method, timeline, locations, evidence handling).
- **Costs**: estimation; fixed-price vs time & material.

### Scope – what does the client want?

- Clients often **unaware** of what they need / can't communicate it.
- Clarify **what** they gain (vulns+risk, fix advice – or just a checkmark / discredit someone) & **why** (real problem? compliance? pre-go-live?).
- Test types: **network, web app, wireless, physical, social engineering**.

- **Channels** to interact: human, physical, wireless, telecom, data networks.
- What's included/excluded? 3rd-party assets affected (check **terms of service**, impact on results)? Use **questionnaires/checklists**.

**Rules of Engagement (RoE)**

- Test method; **timeline**; status meetings (plan/progress/problems).
- **Time of day** to test (e.g. only 20:00–06:00; don't hit backup 00:00–04:00).
- Use of **evasion/stealth** (depends on methodology: black/white-box).
- **Evidence handling**: **encrypt** results, access on **need-to-know**.
- **Permission-to-test document**: states scope, acknowledges **awareness** of tester activities + **all permissions** (customer, 3rd parties); for online tests → **liability** waiver if system crashes despite due care.

**Evidence – sensitive info**

Do **not** store/access **PII**, sensitive PII (religion, race, politics), or **PHI** (health). *But how to prove access was possible without grabbing it?*

**Communication**

Lines of communication; **emergency contacts** (full name/title, authorization to discuss, 2× 24/7 contact, secure large-file exchange); incident-reporting process (target **aware** tests are running → don't wake up mgmt); status report frequency ("ignored customer = former customer"); secure channels.

**Test Methods (OSSTMM) (I)**

Two axes: **attacker's knowledge of target** vs **target's knowledge of attack**:

- **Blind**: ethical hacking, war gaming.
- **Double Blind** = **Black Box** = "classical" pentest (tester knows little, target doesn't know).
- **Gray Box**: vulnerability test.
- **Double Gray Box** = **White Box**.
- **Tandem** = Crystal Box (full mutual knowledge).
- **Reversal** = Red-Team exercise (tester knows all, target knows nothing).

**Challenge – Scope Creep**

**Scope creep** = extension/alteration of scope needing more time & effort. *Ex: client asks "please also verify this is exploitable" / "lower the rating from HIGH to MEDIUM".* ⇒ stick to scope & RoE, re-negotiate formally; don't silently expand or falsify ratings.

05 – PenTesting 2: Intelligence Gathering & Scanning

SWS2 – ZHAW / Tellenbach

Intelligence Gathering (!)

PTES phase 2 (a.k.a. **footprinting / reconnaissance**). Mostly **OSINT** (open-source intel). Feeds the later **vuln analysis & exploitation** phases. *Limitations:* OSINT can be **inaccurate / outdated / manipulated**; some methods (e.g. dumpster diving) out of scope.

3 Methods (by detectability) (!)

- **Passive:** use **3rd-party** data only, never touch the target → **undetectable**. *Ex: Shodan, WHOIS, search engines.*
- **Semi-passive:** **legitimate-looking** queries, stick to normal protocol/traffic, anonymize → should **not** raise alarms. *Ex: normal DNS lookups.*
- **Active:** directly probe target, **should be detected**. *Ex: vuln scanning, port scan, enumeration.*

Intelligence Gathering Levels

- **L1 – Compliance driven:** mostly **automated tools**; minimal effort (a "click-button" baseline).
- **L2 – Best practice:** automated + **manual** analysis; understand the business.
- **L3 – State sponsored:** in-depth **manual**, cultivate social networks; full-scope / red-team.

What to gather

Layered, from outside in: **Physical** (locations) → **Logical** (org structure, business relations) → **Financial** → **Electronic documents** (meta-data!) → **Infrastructure assets** (domains, IPs, tech) → **Employees / HUMINT**.

Browsing the target’s website

Company site, search functions, profile/team pages → names & roles for **social engineering**. Even **removed** content: **Wayback Machine**, **Google cache**, mementoweb.

Search Engine Hacking (Google Dorking)

Craft search queries to surface **exposed** files/pages. **Method: Passive, Level L1/L2.**

Operators (!)

**site:** (limit to domain) • **inurl:** / **allinurl:** • **intext:** / **allintext:** • **intitle:** • **filetype:** (e.g. pdf, xls) • **related:** • **cache:** • **"..."** exact • **-** exclude • **OR** / **|** • **\*** wildcard • **..** number range • **AROUND(n)** proximity

GHDB & automation

- **Google Hacking Database (GHDB)** on **exploit-db.com**: ready-made dorks by category (login portals, error messages, files with passwords...).
- Tool: **Google Hacking Diggity**.
- **Automation is hard:** throttling, **CAPTCHAs** ("I'm not a robot" after suspicious traffic), query limits/cost; dorks have short **lifetime**.

Infrastructure Assets

Start from a **URL or company name**. Relevant: **domains, IP addresses & network provider(s), external infra profile, (defence) technologies** used.

Finding Domains

**Problem:** no **public register** of all domains. For **gTLD** (.com/.net/.org) you can get **zone files** (lists 2nd-level domains); for **ccTLD** only a few countries publish them. **Goal:** find all domains (= potential **entry points**) owned by the company / an employee, or by **B2B** partners.

Approaches (depend on what you know)

Company/employee name or email • a set of IPs • a domain (find subdomains) • a domain (find related domains).

From company/employee → WHOIS (!)

- **WHOIS** = protocol to query the **registered owner / assignee** of an Internet resource (domain, IP block, AS).
- **RIRs** (Regional Internet Registries) run WHOIS servers: **ARIN** (N.America), **RIPE NCC** (Europe), **APNIC** (Asia-Pac), **LACNIC** (Latin America), **AfriNIC** (Africa).
- Entries are **cross-referenced** (ARIN query for a RIPE record returns a pointer to RIPE).
- No standard to find the **responsible** server → manual: start at **whois.iana.org** → query TLD (e.g. **ch** → org = **SWITCH**, server whois.nic.ch) → query that org’s DB.
- Query via **whois** CLI or web front-ends (e.g. nic.ch/whois for .ch).

Reverse WHOIS

WHOIS normally: **domain → owner**. Usually **no official reverse lookup** (name/email → domains), and registrations may be **privacy-protected (GDPR)**. → use 3rd-party: **viewdns.info, whoisxmlapi.com**. Caveat: may be **incomplete** or list **too many** domains.

How useful is WHOIS? (!)

- **Domain WHOIS = often thin:** since **GDPR** the owner fields are **redacted** / hidden behind a privacy proxy or the registrar (you see "switchplus AG", not the real holder). Still get: **name servers, registrar, dates**.
- **IP / AS WHOIS = still gold:** **not** redacted (GDPR protects people, not netblocks) → querying one IP returns the **whole owned range (inetnum)** + org + AS. Bigger prize than one domain’s owner field.

From a set of IPs → DNS (PTR) (!)

DNS forward = **name → IP** (A record). **PTR record** = the **reverse, IP → hostname**, stored in the **in-addr.arpa** zone (IP written backwards):

- **A:** www.zhaw.ch → 160.85.104.96
- **PTR:** 96.104.85.160.in-addr.arpa → wwwr1.zhaw.ch

**Not mandatory:** every live host needs an A record, but the IP owner need not publish a PTR (or it’s a generic provider name, e.g. ec2-..amazonaws.com).

**Use in recon – reverse-DNS sweep:** once WHOIS/BGP told you the company **owns** a block, PTR-lookup **every IP** in it to turn "they own this /16" into a **list of named, live machines**:

- 160.85.104.96 → wwwr1.zhaw.ch
- 160.85.104.121 → srv-mail-011.zhaw.ch
- 160.85.104.245 → srv-app-303-data2.zhaw.ch

Names **leak the internal map:** srv-mail==mail, srv-app==app, ns1/ns2=DNS. A PTR existing ≈ IP is in service. Tool: **robtex.com** (shows A record + all names on the same IP → related domains).

Subdomains of a domain

**NO official way** (would need the **authoritative name server**’s data). Combine **unofficial** methods:

- **Web-page scraping** (crawl main domain, grab subdomains).
- **Search engines** (design queries).
- **Brute-force** enumeration (mail., dev., ...).
- **X.509 certs:** read **Subject Alternative Name (SAN)** – one cert often lists many (sub)domains.

Related domains

Domains on the **same IP** • same **NS** (own NS → few & relevant; provider’s NS → many irrelevant) • same **MX** (mailserver) • relations from web-page data. 3rd-party combine sources (whoisxmlapi, findsubdomains, spyse). **Method: passive (WHOIS/SE/3rd-party) + active (DNS), L1/L2.**

Finding IP Addresses

Company OWNS IP blocks / its own AS

- IP ranges assigned by **RIRs**; info includes contact + the **Autonomous System (AS)** the block belongs to.
- **IP → network prefix:** RIR websites / WHOIS (e.g. RIPE query for an IP → **inetnum** range like 160.85.0.0–160.85.255.255).
- **Company name → "all" IP ranges:** **BGP Toolkit (bgp.he.net, Hurricane Electric)** – shows prefixes (v4 & v6) per AS. Needed because enumerating IPv6 by brute force is infeasible → use **BGP** routing data.

Company does NOT own blocks/AS

- **Domains → IPs:** **Forward-DNS** (host IP + NS/MX servers).
- **IP → more IPs:** **Reverse-DNS** on the network block + other checks.
- **Search services** (by company name): **Search Engine Hacking** (web-only, incomplete), **Shodan / Censys**.

DNS tools

- **nslookup <IP>:** reverse → in-addr.arpa → hostname.
- **dig ns <domain>:** name servers; **dig mx <domain>:** mailservers (e.g. zhaw → outlook.com).
- **Reverse-DNS sweep:** script PTR lookups over a whole **/16** (class B) to harvest active hostnames. **Is this active or passive?** → **passive** (you query **DNS**, not the target’s own systems). **Method: passive (RIR/3rd-party), semi-active/active (DNS), L1/L2/(L3).** Passive discovery is **incomplete** → verify.

Scanning

Take recon results and examine networks & hosts **in detail**. Goals: determine **network structure**, find **reachable hosts** (depends on tester location – inside/outside), and **analyse hosts** (OS, services, software → hints at vulns).

Network structure

- Reveals interesting **areas** (e.g. a **DMZ**) to prioritize.
- Provides **attack paths:** a host you can’t reach directly may be reached via a **stepping stone** in its network.
- **traceroute:** lists all hops to target. Default **UDP or ICMP (-I)**; **tcptraceroute** uses TCP → try both, esp. through firewalls.

*Deductions from a trace:* two hosts behind the same router are **likely** same network; IPs 104.96 & 104.121 ⇒ at least a **/25**. An **asterisk** = no answer ⇒ **"some filtering"** (firewall) at that hop. Scan also **internal→internal** and **internal→external** for more structure.

hping3

More flexible than traceroute: pick **protocol (ICMP/UDP/TCP)** & **dest port**. *Ex: TCP port 80 SYN traceroute – port 80 gets through firewalls* more often than UDP/ICMP.

nmap (!)

Host discovery (ping scan) (!)

**nmap -sn <target>** = "scan, **no** port scan". Answers **one** question: **is the host alive?** – then stops (no port table). (*old syntax: -sP*). First resolves the name to an IP, then probes it. It doesn't send just **one** ping – it fires a **bundle** and counts the host as **up** if **anything** replies (firewalls block some probe types, so it tries several):

- **ICMP echo** – the classic ping.
- **TCP SYN → 443** – SYN/ACK back = alive.
- **TCP ACK → 80** – RST back = alive (slips past some firewalls).
- **ICMP timestamp** – backup when echo is blocked.

**Local network:** switches to **ARP** requests (can't reach a LAN IP without its MAC; ARP can't be L3-filtered).

Mental model:

- **-sn** = knock, see who's home, leave (discovery only) → then port-scan only those that answered.
- **-Pn** = skip the knock, assume **everyone** is home, port-scan all. Slower, but **pentest-safe**: a host that ignores all discovery probes is marked "down" & **skipped** by default → -Pn so no live host slips through.
- **default** (neither) = knock first, then scan whoever answered.

Scan techniques (!)

- **-sS** TCP SYN: **default**, fast, stealthy = **half-open**.
- **-sT** TCP Connect: uses OS **connect()**; slower, **more detectable** (no raw packets / no root).
- **-sA** TCP ACK: test **firewall rulesets**.
- **-sU** UDP: much slower than TCP.
- **-sN/-sF/-sX** Null/FIN/XMAS: odd flag combos to slip past **stateless** firewalls.
- **-sO** IP protocol: which IP protocols the target supports.

Port discovery

**IANA ranges:** well-known 1–1023, registered 1024–49151, dynamic 49152–65535. Selection: **-p** (all), **-p22,53, -p20-25**, default = **top 1000**, **-F** = top 100, **–top-ports 10**. Combine: **-sU -sS -p U:53,T:21,80**. *Effectiveness:* top-1000 TCP ≈93%, top-1017 UDP ≈100% → **fewer ports** already catch most services.

Service / version detection

**-sV** (in **-A**): interrogate open ports with **service probes**. Uses the **nmap-service-probes** DB of probes + **match expressions**; mostly parses the **banner** (connect response). Plain port label can lie (apps on arbitrary ports) → **-sV** gives real product+version. Tip: **–reason** = why a port is open/closed/filtered.

OS detection

**-O -v:** sends up to **16 TCP/UDP/ICMP probes** exploiting **RFC ambiguities**; combines response attributes into an OS **fingerprint**.

Get it all: -A

**-A** = OS detection + version detection + **script scanning** + traceroute. Yields protocols, app names, versions, hostname, device type, OS family, **CPE**.

Manual banner grabbing

Also good for versions: **nc/telnet** (e.g. ssh banner "SSH-2.0-OpenSSH..."), **openssl s\_client** for TLS. *Ex: server header "BigIP" ⇒ a WAF (F5) sits in front.*

Passive scanning

**Shodan** (shodan.io) – search engine for **Internet-connected devices**: scans the net, grabs **banners**; historic data; free tier limited. **Censys** (censys.io) – similar, more **frequently updated**, only **current** data. Both queried by net/company → hosts, ports, products, certs (SAN names).

Footprinting Defences

Find the **defensive systems** (more attack vectors + know how to **evade detection**): firewalls, **WAF**, IDS, anti-virus.

- Brute-force protections, (D)DoS misuse, auth type/protocol.
- **MFA** ⇒ attack vector must also obtain the **2nd factor**.
- **Federated** systems ⇒ attack via other **members of the federation**.
- **Active:** inspect banners/HTTP responses, device fingerprinting, social engineering. **Passive:** search-engine hacking, Censys/Shodan.

*Ex WAF:* manual = inspect HTTP response (Server: BigIP); automated = **nmap http-waf-fingerprint** ("Detected WAF: F5 BigIP").

HUMINT

Human intelligence = **direct interaction: physical** (observation) or **verbal** (call/chat/on-site). First learn all you can about a person: **online accounts**, social-media posts. **Automate account discovery:** sites that tell if a **username exists** on a service (the service must **leak** existence – many do via login or **password-reset**). Tools: **knowem.com**, **checkusernames.com**.

**Take-away:** Intelligence gathering is the **first** pentest step. After it you know domains, IP ranges, technical contacts, hostnames/IPs – and sometimes critical info **voluntarily leaked** by employees. Scanning then maps the network & hosts (OS, services, versions, likely vulns) using **traceroute, nmap, Shodan/Censys**.